Bootstrapping Vision-Language-Action Models for New Robot Embodiments with Staged Reinforcement Learning

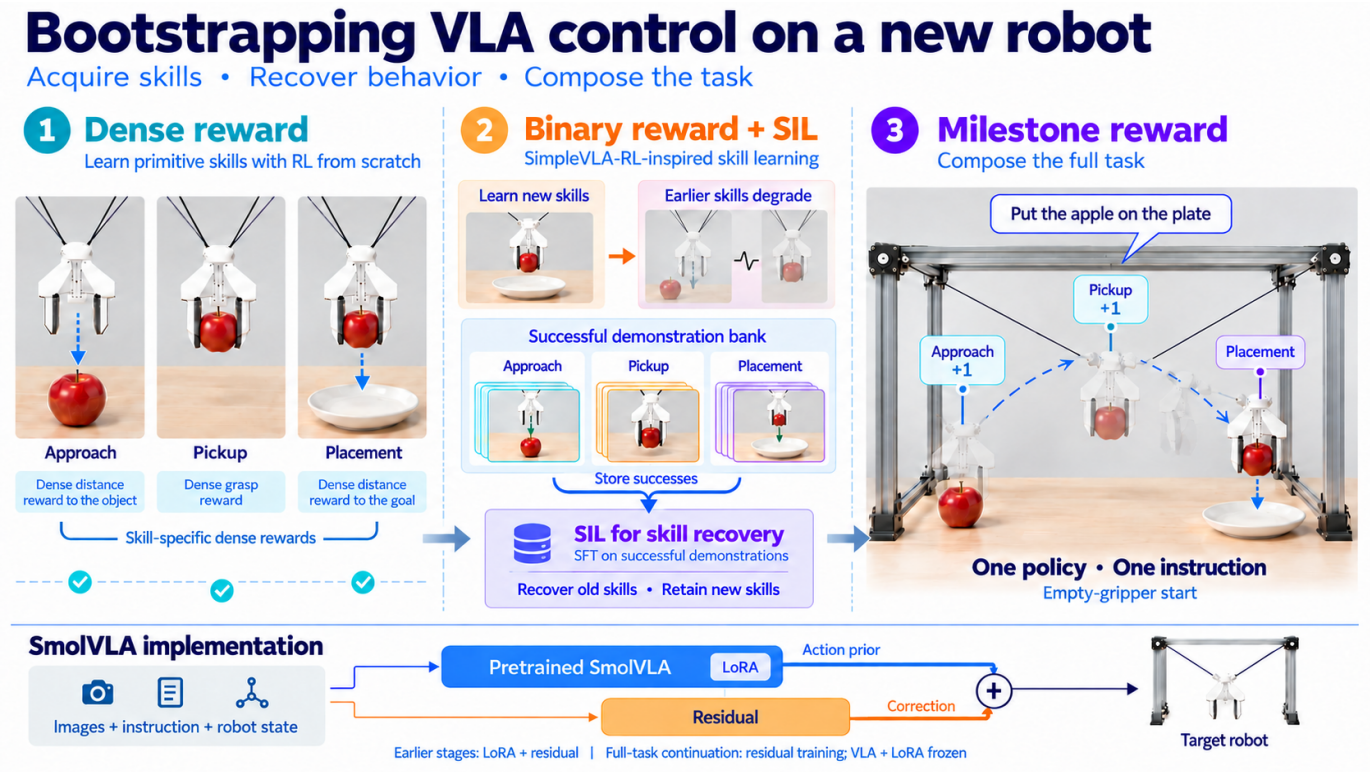


Fig. 1. Proposed pipeline and SmolVLA implementation. Dense feedback acquires skills; binary refinement and balanced self-imitation consolidate them; milestone-based RL learns the complete task under one instruction. Skill-specific resets are used during acquisition. Full-task evaluation starts with an empty gripper and uses no external switching between specialists.

WORKING DRAFT — 29 October 2026

***Abstract*—A pretrained vision-language-action (VLA) model can produce few successful rollouts when transferred to a new action interface. Sparse rewards then hinder learning, while sequential skill training can erase earlier behavior. We propose a pipeline that uses dense rewards only at the beginning to acquire a base policy for simple skills. Binary refinement and balanced self-imitation consolidate these skills, followed by staged reinforcement learning for the complete multi-stage task. Binary milestones connect the skills without requiring a continuous progress function for every phase of the full task. We compare feedback variants with OpenVLA-OFT on LIBERO-Spatial and test adaptation with a SmolVLA residual controller and LoRA on the CDPR. On LIBERO-Spatial, combined dense and phase feedback reaches 65.3% success versus 61.7% with terminal-only reward at the same update budget. The CDPR controller achieves 70.61% strict success in simulation.**

***Index Terms*—Robot learning, reinforcement learning, manipulation, vision-language-action models, robot adaptation.**

I. Introduction

Vision-language-action (VLA) models provide useful visual and language representations for robot control. Transferring these representations to a new robot still requires learning how its actions affect the scene. If the initial policy rarely completes the task, binary success provides little guidance for reinforcement learning.

Consider *put the apple on the plate*. A failed rollout may contain a correct approach and grasp followed by a poor release. Treating the entire attempt as a failure discards this distinction. Training the parts separately helps exploration, but a policy can forget earlier skills while learning later ones. It must then connect the retained skills into a complete behavior.

We address these problems through the pipeline in Fig. 1. Dense rewards are used only at the beginning to acquire a base policy for simple skills. Binary refinement then targets skill completion, while balanced self-imitation retains successful behavior across instructions. Finally, staged RL trains this policy to execute the complete multi-stage task from an empty-gripper start, using binary milestones throughout full-task training.

The pipeline separates feedback design and skill consolidation from the robot-specific action mapping. We use OpenVLA-OFT [1] on LIBERO-Spatial to evaluate inter-

mediate feedback for full-task composition. We then assess skill retention and complete-task adaptation in cable-driven parallel robot (CDPR) simulation using SmolVLA [2], LoRA, and a residual action controller. These experiments examine connected parts of the same adaptation procedure. The ongoing WX200 evaluation extends the approach to a 5-DoF serial manipulator.

We contribute a training schedule that combines initial dense acquisition of simple skills, binary refinement with instruction-balanced self-imitation, and staged full-task learning under one instruction. We specify milestone credit that retains local credit for achieved approach and pickup stages when later placement fails. Our experiments compare intermediate and terminal feedback for composition, measure skill recovery after sequential training, and report complete-task performance after action-interface adaptation.

II. Related Work

Group-relative RL and dynamic sampling. GRPO estimates policy advantages by comparing returns within groups of sampled trajectories [3]. DAPO introduces dynamic sampling that filters groups with uniform binary outcomes and collects additional samples to maintain an informative training batch [4]. SimpleVLA-RL adapts group-relative optimization to interactive robot rollouts, using dynamic sampling and exploration enhancements to improve task execution and discover behaviors beyond the supervised demonstrations [5]. These methods provide a basis for learning task-specific behavior through interaction. We use DAPO-style group filtering and resampling during skill learning.

Alternating RL and supervised learning. iRe-VLA alternates online RL on a lightweight action head with a frozen vision-language backbone and supervised updates using expert data together with successful online trajectories [6]. This separates exploration from supervised consolidation while allowing the large model to benefit from collected experience. Our self-imitation step fits an instruction-balanced mixture of earlier and current successes to retain skills during sequential training.

Residual RL and distillation. Probe, Learn, Distill (PLD) trains lightweight residual specialists over a frozen VLA, collects trajectories through policy-aligned residual interventions, and distills the resulting data into the generalist [7]. We adopt the residual-learning principle to adapt the target action interface. Our training procedure links skill acquisition and successful-experience consolidation before learning to execute the complete instruction.

III. Methods

The goal is to adapt a pretrained VLA to complete language-conditioned manipulation tasks on a target robot. We study multi-stage pick-and-place tasks: an instruction such as *put the apple on the plate* requires approach, grasp, lift, transport, and release. The policy receives camera images, proprioception, and an instruction, predicts an action chunk, and executes a prefix before observing again. We evaluate complete-task success independently of the reward used for training.

During acquisition, each skill has its own instruction, success condition, and starting states. For example, placement can begin with an object already held. During composition and final evaluation, the robot starts with an empty gripper and receives one complete-task instruction. This distinction prevents isolated skill performance from being treated as full-task success.

A. Dense Acquisition, Binary Refinement, and Self-Imitation

We use dense rewards only at the beginning of adaptation to learn a base policy for simple skills. Each skill has a distinct instruction and reset distribution:

- **Approach:** *move to <object>*. Reach the named object with an empty gripper, without requiring a grasp.
- **Pickup:** *pick up <object>*. Learn to grasp and lift the named object.
- **Placement:** *put <object> into bowl* or *put <object> on the plate*. Start with the object already grasped and learn transport and release at the named destination.

The object placeholder is replaced by its name. These instructions condition the policy on the requested skill and target; the instruction remains attached to each retained trajectory. Distance reduction and manipulation events guide initial acquisition. We then refine the skills with binary success rewards and consolidate them through self-imitation. The resulting base policy must retain all component skills before full-task learning.

Successful trajectories are retained by instruction. When later training degrades earlier skills, self-imitation fits a balanced mixture of earlier and current successes:

$$\mathcal{L}_{\mathrm{SI}} = \mathbb{E}_{\mathrm{success}} \left[\|\hat{a} - a\|^2 \right]. \tag{1}$$

Here $\hat{a}$ is the predicted action and a is the recorded executed action. The average balances instructions and excludes unused action slots. Policy-dependent features are refreshed before fitting. Balancing matters because imitating only the newest skill can reinforce forgetting.

Algorithm 1 summarizes the schedule. Progress is checked with fixed skill evaluations. After an initial training period, automated stages can end at a success target or a plateau; budget exhaustion also ends a stage. The CDPR sequence used validation-guided manual transitions. After consolidation, all learned skills are evaluated again. Automated runs retain the consolidation update only when recovery improves the degraded skills on average and keeps other skills within tolerance.

B. SmolVLA with an Embodiment-Specific Residual

Figure 2 shows one implementation of the pipeline. Following the residual-learning ingredient of Probe, Learn, Distill (PLD) [7], we retain a pretrained expert and adapt its actions with a lightweight residual. The SmolVLA expert supplies an action prior, mapped to the target control interface. In our five-channel controller, eight action slots form a 40-dimensional prior. A frozen visual projection supplies 512 features; six proprioceptive values complete the 518-dimensional observation.

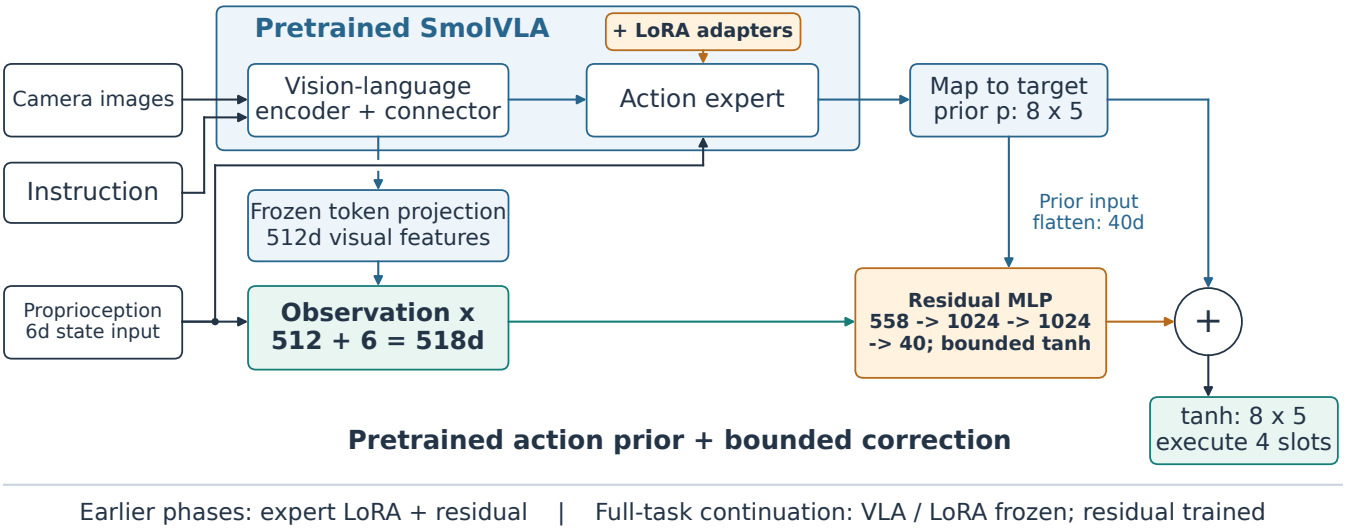


Fig. 2. SmolVLA implementation of the pipeline. The added residual combines 518 observation features with a 40-dimensional action prior. LoRA adapts the pretrained expert during earlier training; the evaluated full-task continuation freezes the VLA and LoRA weights. Feature and action dimensions refer to the five-channel controller.

Algorithm 1 Staged VLA adaptation

Require: Pretrained VLA, simple-skill instructions, fixed evaluations
Initialize the target action controller and success banks
for reward mode: dense acquisition, then binary refinement **do**
  for skill: approach, pickup, placement **do**
    repeat
      Collect same-scene rollout groups for the skill
      Update with $\mathcal{L}_{\mathrm{RL}}$; retain successful trajectories
      Evaluate skill success
    until target, plateau, or stage budget ends training
    Evaluate retention of all learned skills
    if earlier skills have degraded **then**
      Balance earlier and current successful experience
      Fit $\mathcal{L}_{\mathrm{SI}}$; validate recovery and retention
    end if
  end for
end for
Initialize full-task learning from the consolidated base policy
repeat
  Collect full-task rollouts from empty-gripper starts
  Assign milestone credit; protect achieved prefix skills
  Update with $\mathcal{L}_{\mathrm{RL}}$; evaluate complete-task success
until target, plateau, or full-task budget ends training
if composition plateaus and budget remains **then**
  Freeze VLA/LoRA; retain the bounded residual
  Reset optimizer; keep milestone rewards unchanged
  repeat
    Collect full-task groups; execute clipped commands
    Store pre-clipping samples and episode offsets
    Score samples conditional on their stored offsets
    Assign milestone credit; protect achieved prefix skills
    Update residual and exploration with $\mathcal{L}_{\mathrm{RL}}$
    Evaluate complete-task success
  until target, renewed plateau, or budget exhaustion
end if
return One policy for the complete instruction

The residual receives this observation and the 40-dimensional prior, and predicts a bounded correction. The combined action is bounded before execution.

LoRA adapts the action expert during earlier training. In the evaluated full-task continuation, the inherited VLA and LoRA weights are frozen while the residual and exploration parameters are trained. The backbone, action mapping, and feature dimensions can change with the embodiment; the training schedule remains the same.

C. Milestone-Based Full-Task Composition

After skill consolidation, the base policy learns the complete task from empty-gripper starts using binary milestone rewards. The full-task prompt is *put <object> into <destination>*, where the destination is plate or bowl. This includes the plate wording *into plate*, whereas the isolated plate-placement skill uses *on the plate*. The full-task instruction stays fixed throughout approach, pickup, and placement. Each milestone marks completion of part of this sequence. Continuous dense shaping is confined to the initial simple-skill acquisition in Section III-A. The LIBERO reward comparison examines how intermediate feedback can support execution of a complete instruction.

We consider terminal-only, phase-only, and combined dense-and-phase rewards. The combined reward uses a manually defined sequence of reach, grasp, lift, move, and place phases, with a hand-specified progress function for each phase.

At each simulator step, the tracker uses object and gripper state to compute progress for the active phase. Reaching progress increases as gripper–object distance decreases; grasping combines proximity with gripper closure and grasp state; lifting uses height gained above the initial object position; transport uses planar object–target distance; placement uses target proximity, release, and environment success. Each progress measure is normalized to the interval from zero to one. For progress p_t and the phase's previous maximum p_{best},

the rewarded increment is $\max(p_t - p_{\text{best}}, 0)$. A completion predicate advances the tracker to the next phase and produces a phase bonus. The weighted progress increment and phase bonus are bounded and combined with terminal feedback. The native task-success verifier remains unchanged.

In the phase-only condition, the continuous progress weight is zero, leaving phase-completion bonuses and terminal feedback.

The dense construction requires a progress function, normalization, and relevant state variables for every phase. Extending it to a new multi-stage task therefore requires further task-specific reward design. Our adaptation pipeline uses this detailed feedback only to obtain the initial base policy on simple skills. Full-task training uses binary milestones and their completion predicates, so the composition stage does not require a continuous progress function for every part of the instruction. The motivation follows from a simple group calculation. For G independent rollouts from one scene, each with success probability p, the probability of observing both success and failure is

$$P_{\text{mixed}} = 1 - p^G - (1-p)^G. \tag{2}$$

With eight rollouts, this probability is only 21.6% at 3% success, but 94.2% at 30% success. Milestones can therefore provide informative comparisons before complete-task success becomes common.

In the adapted CDPR controller, approach, held pickup, and final placement serve as milestones for assigning credit within a rollout. The policy executes the complete instruction without an external controller switching between specialists.

The main rule is simple: a later placement failure should not remove the local credit for a successful approach or pickup. We compare milestone outcomes within groups of rollouts from the same scene. Actions receive credit for the milestone they contribute to, while full-task success provides an additional preference for completing the chain. A negative completion bonus is suppressed on already achieved approach and pickup milestones. Placement receives its completion credit once.

The rule retains the assigned credit for completed milestones while training the remaining task.

D. Stabilized Updates and Full-Task Continuation

We use a clipped group-relative policy loss with entropy regularization:

$$\mathcal{L}_{\text{RL}} = \mathcal{L}_{\text{clip}} - \eta\mathcal{H}. \tag{3}$$

The clipped loss compares rollouts within each scene group and limits the incentive for large changes in action likelihood. We use separate upper and lower clipping bounds. The entropy term $\mathcal{H}$, weighted by η, maintains exploration during manipulation and composition. Together, these choices help keep early updates conservative while successful behavior is still rare.

During skill learning, DAPO-style dynamic sampling filters uninformative groups and collects additional rollout groups within the configured collection budget [4]. With binary rewards, uniform outcomes provide no group-relative contrast. During dense acquisition, filtering instead checks within-group return variation, so partial progress can provide a learning signal before a group contains a successful skill execution.

During full-task continuation, the inherited VLA and LoRA remain frozen while the residual and exploration scale are trained with a fresh optimizer and the same milestone rewards. The controller adds Gaussian exploration noise and an episode-level offset to its predicted mean before clipping the command for execution. The offset is sampled once per episode and applied during selected phases. We store the original sample and the applied offset, then use both to evaluate its likelihood under the old and new policies [8].

Let $\mu_{\theta,t}$ denote the controller's predicted mean, σ_θ the learned exploration scale, and b_t the applied offset. The sampled command u_t and executed action a_t are

$$\begin{aligned} u_t &\sim \mathcal{N}(\mu_{\theta,t} + b_t, \operatorname{diag}(\sigma_\theta^2)), \\ a_t &= \operatorname{clip}(u_t, -1, 1). \end{aligned} \tag{4}$$

The likelihood ratio scores the stored u_t conditional on the same b_t. Section IV-C reports complete-task performance.

IV. Experiments

We first evaluate skill retention during base-policy learning in CDPR simulation, then intermediate feedback for composition on LIBERO-Spatial, and finally complete-task CDPR adaptation. The ongoing WX200 evaluation extends the approach to another embodiment.

The CDPR simulation uses MuJoCo Warp/MJ-Lab, four object categories (apple, orange, potato, and tomato), and plate or bowl destinations. The action interface controls Cartesian translation, yaw, and the gripper.

A. Base-Policy Skill Learning and Retention

These measurements concern dense-reward training of simple CDPR skills and their consolidation into a base policy, before full-task composition. Each skill uses the instruction and reset protocol in Section III-A; placement tests start with the object already held. Learning a new skill can degrade previously learned behavior, although the same policy must retain all component skills to execute the complete multi-stage task.

In a forgetting evaluation, mean reaching success falls from 64.1% for the reaching reference to 5.3% after placement RL, over three evaluation rounds per condition. The recovery evaluation in Fig. 3 is a separate measurement: balanced self-imitation raises reaching success from 8.0% to 31.1%. Plate placement changes from 50.0% to 52.3%, and bowl placement from 23.05% to 27.65%. The bank combines earlier reaching successes with current placement successes, balancing the retained data by instruction.

Recovery is partial, with placement performance retained while reaching improves. These measurements cover approach and placement and do not isolate the contribution of self-imitation to final full-task success.

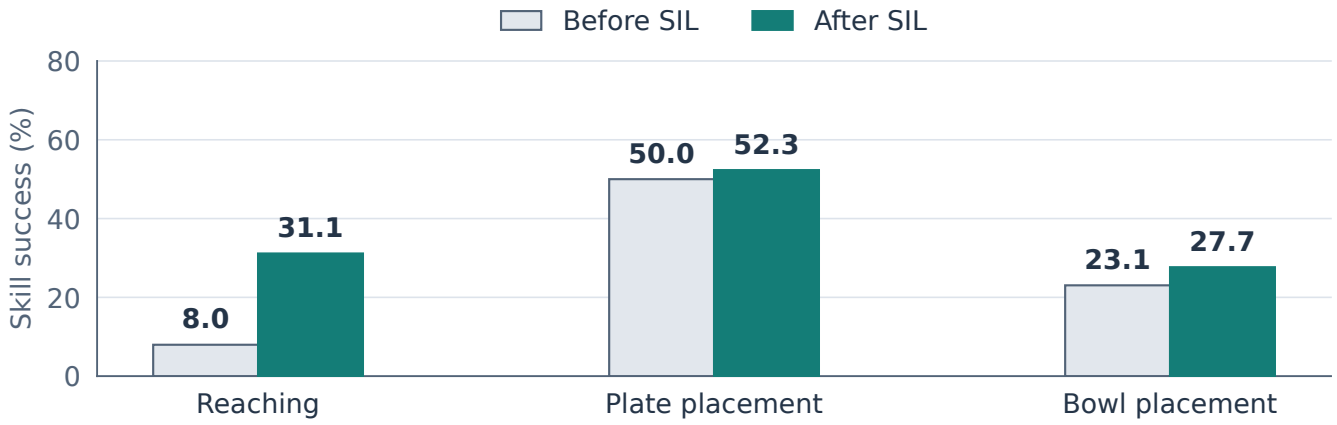


Fig. 3. Base-policy skill recovery before full-task composition. Bars show means across three evaluation rounds for reaching and two rounds for each placement skill, per condition. Reaching uses the instruction *move to <object>*; placement starts with the object already held. Reaching partially recovers while both placement means increase.

B. Intermediate Feedback for Full-Task Composition

We fine-tune an OpenVLA-OFT-based policy with SimpleVLA-RL and LoRA on complete instructions for the ten LIBERO-Spatial tasks [1], [5], [9], using the feedback variants in Section III-C. All runs share a 61.0% initialization and use 300 validation rollouts per endpoint. The dense-and-phase and terminal-only conditions use terminal verifier coefficient 5 and a 160-update budget. The phase-only condition uses phase-completion bonuses with terminal verifier coefficient 1 and a 110-update budget. Dense-and-phase feedback reaches 65.3% success versus 61.7% for terminal-only SimpleVLA-RL (Table I). Phase-only feedback reaches 62.3%, improving on the 61.0% initialization using completion events as intermediate feedback.

For the adaptation pipeline, we use dense rewards to acquire the initial base policy on simple skills, then train the complete multi-stage task with staged reward. Milestone completion supplies intermediate credit without requiring a continuous progress function for every phase of each new instruction.

TABLE I
LIBERO-SPATIAL REWARD COMPARISON FROM A 61.0% INITIALIZATION. SUCCESS IS REPORTED IN PERCENT; Δ IS THE GAIN IN PERCENTAGE POINTS.

Configuration	Updates	Success	Δ
Terminal only (SimpleVLA-RL [5])	160	61.7	+0.7
Dense + phase	160	65.3	+4.3
Phase-only	110	62.3	+1.3

C. SmolVLA: Complete-Task Adaptation

Strict success requires grasp, held lift, intentional release, and correct placement, without carry slip or wrong-placement events.

The development validation comprises 1,024 full-task episodes: 512 plate and 512 bowl attempts. Each attempt starts with an empty gripper and uses one full-task instruction, without teacher assistance. All attempts, including numerical failures, remain in the denominator. The adapted controller completes 723 of 1,024 development validation attempts (70.61% strict success; Table II). Physical grasp occurs in 92.48% of attempts and held lift in 88.28%. Completion rates are 393/512 (76.76%) for plates and 330/512 (64.45%) for bowls. Carry slips (16.70%) and wrong placements (14.16%) remain the main failure modes.

TABLE II
CDPR COMPLETE TASK DEVELOPMENT EVALUATION.

Outcome	Attempts (%)
Physical grasp	92.48
Held lift	88.28
Strict completion (723/1,024)	70.61
Carry slip	16.70
Wrong placement	14.16

D. Additional Embodiment: WX200

An ongoing experiment applies the pipeline to the five-DoF WX200 serial manipulator. This setting changes the robot mechanism while retaining the skill-learning and composition schedule. Figure 4 reserves the setup and rollout illustrations. Quantitative success rates will be added after the evaluation is complete.

E. Scope of the Evidence

The LIBERO comparisons evaluate reward variants on complete manipulation instructions. In the adaptation pipeline, dense rewards are used only for initial simple-skill acquisition; the CDPR full-task controller is trained with binary milestone feedback. The complete-task result comes from one SmolVLA training lineage and a development validation panel used during training. Retention measurements cover a small number of rounds. Earlier assisted training is part of the inherited CDPR policy; the reported full-task evaluation uses no assistance.

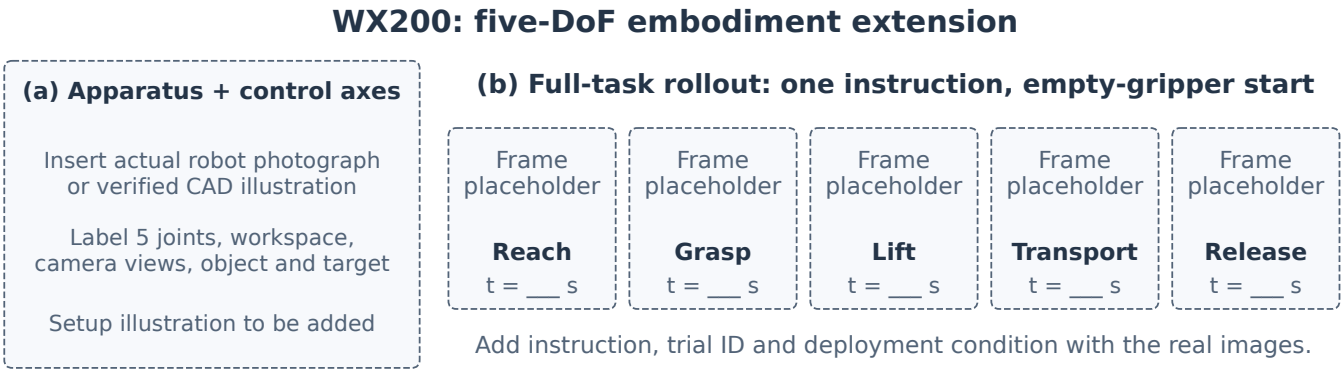


Fig. 4. WX200 evaluation layout. (a) Apparatus, workspace, cameras, and control axes. (b) Reach, grasp, lift, transport, and release under one instruction. The current panels reserve space for setup images and timestamped rollout frames from the ongoing experiment.

V. Conclusion

We presented a VLA adaptation pipeline that uses dense rewards only for initial acquisition of a base policy on simple skills. Binary refinement and self-imitation consolidate this policy before staged RL learns the complete multi-stage task. The LIBERO-Spatial reward comparison examines intermediate feedback, and binary milestones provide the composition stage with completion-based credit. Skill evaluations show partial recovery of earlier behavior after balanced self-imitation, and the adapted CDPR controller reaches 70.61% strict success in simulation development validation.

VI. Future Work

We will complete the WX200 evaluation to test the pipeline on another embodiment. Further experiments will compare training stages under matched budgets, repeat training across independent seeds, and evaluate unseen scenes. Automatic phase detection and progress estimation could also reduce the need for manually designed reward predicates.

References

[1] M. J. Kim, C. Finn, and P. Liang, "Fine-tuning vision-language-action models: Optimizing speed and success," *arXiv preprint arXiv:2502.19645*, 2025. [Online]. Available: https://arxiv.org/abs/2502.19645

[2] M. Shukor, D. Aubakirova, F. Capuano, P. Kooijmans, S. Palma, A. Zouitine, M. Aractingi, C. Pascal, M. Russi, A. Marafioti, S. Alibert, M. Cord, T. Wolf, and R. Cadene, "SmolVLA: A vision-language-action model for affordable and efficient robotics," *arXiv preprint arXiv:2506.01844*, 2025. [Online]. Available: https://arxiv.org/abs/2506.01844

[3] Z. Shao, P. Wang, Q. Zhu, R. Xu, J. Song, X. Bi, H. Zhang, M. Zhang, Y. K. Li, Y. Wu, and D. Guo, "DeepSeekMath: Pushing the limits of mathematical reasoning in open language models," *arXiv preprint arXiv:2402.03300*, 2024. [Online]. Available: https://arxiv.org/abs/2402.03300

[4] Q. Yu, Z. Zhang, R. Zhu *et al.*, "DAPO: An open-source LLM reinforcement learning system at scale," *arXiv preprint arXiv:2503.14476*, 2025. [Online]. Available: https://arxiv.org/abs/2503.14476

[5] H. Li, Y. Zuo, J. Yu, Y. Zhang, Z. Yang, K. Zhang, X. Zhu, Y. Zhang, T. Chen, G. Cui, D. Wang, D. Luo, Y. Fan, Y. Sun, J. Zeng, J. Pang, S. Zhang, Y. Wang, Y. Mu, B. Zhou, and N. Ding, "SimpleVLA-RL: Scaling VLA training via reinforcement learning," *arXiv preprint arXiv:2509.09674*, 2025. [Online]. Available: https://arxiv.org/abs/2509.09674

[6] Y. Guo, J. Zhang, X. Chen, X. Ji, Y.-J. Wang, Y. Hu, and J. Chen, "Improving vision-language-action model with online reinforcement learning," *arXiv preprint arXiv:2501.16664*, 2025. [Online]. Available: https://arxiv.org/abs/2501.16664

[7] W. Xiao, H. Lin, A. Peng, H. Xue, T. He, Y. Xie, F. Hu, J. Wu, Z. Luo, L. Fan, G. Shi, and Y. Zhu, "Self-improving vision-language-action models with data generation via residual RL," *arXiv preprint arXiv:2511.00091*, 2025. [Online]. Available: https://arxiv.org/abs/2511.00091

[8] Y. Fujita and S.-i. Maeda, "Clipped action policy gradient," in *Proceedings of the 35th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 80, 2018, pp. 1597–1606. [Online]. Available: https://proceedings.mlr.press/v80/fujita18a.html

[9] B. Liu, Y. Zhu, C. Gao, Y. Feng, Q. Liu, Y. Zhu, and P. Stone, "LIBERO: Benchmarking knowledge transfer for lifelong robot learning," in *Advances in Neural Information Processing Systems*, vol. 36, 2023. [Online]. Available: https://arxiv.org/abs/2306.03310